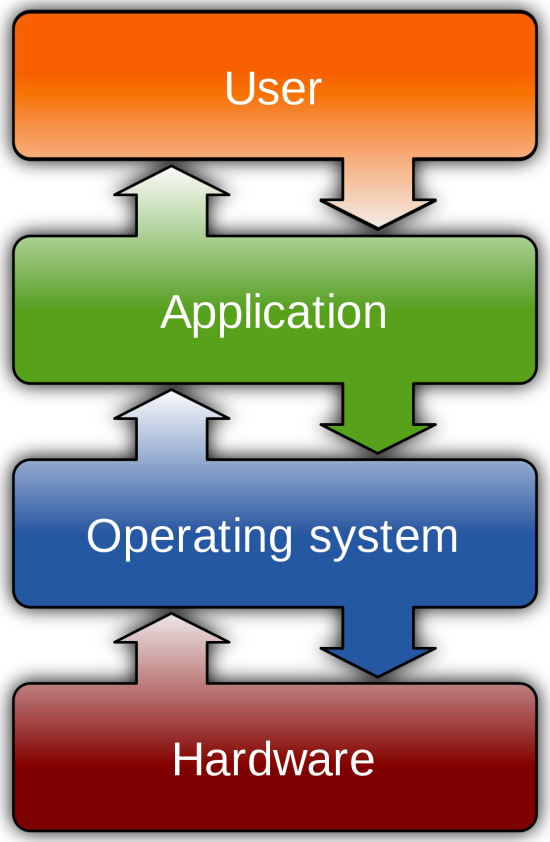
Operating Systems & Parallelism

DSC 102: Systems for Scalable Analytics

Haojian Jin

UC San Diego

Where we are



- **L1:** the hardware: processors, caches, DRAM, disks
- **L2:** the agent above the API, and its memory
- **Today:** the layer in between, the **operating system**, which turns one machine into many running programs

One CPU, many programs: how?

Five questions for today

1. What is an **operating system**, and why do we need one?
2. What is a **process**?
3. How does **one CPU run many processes**?
4. Which process should **run next**?
5. What changes when one chip has **many cores**?

Q1 · What is an OS, and why do we need one?



Many orders, a few cooks, one stove, one sink.

- Who gets the stove next?
- What happens when two cooks reach for the same pan?
- How does a cook wait for the oven without blocking the others?

A computer has the same problem: many programs, scarce hardware.

Screenshot: Overcooked (Ghost Town Games / Team17).

An OS is like a government



An OS is a large set of interrelated programs that make it easier for applications to use the hardware **effectively, efficiently, and securely**.

- Without one, every program would speak machine code to the disk and the network card
- Legislature, executive, judiciary: separate powers that each do one job well

Two principles behind any OS (or any system)

Modularity

Divide the system into functionally cohesive components that each do their job well.

The executive–legislature–judiciary split.

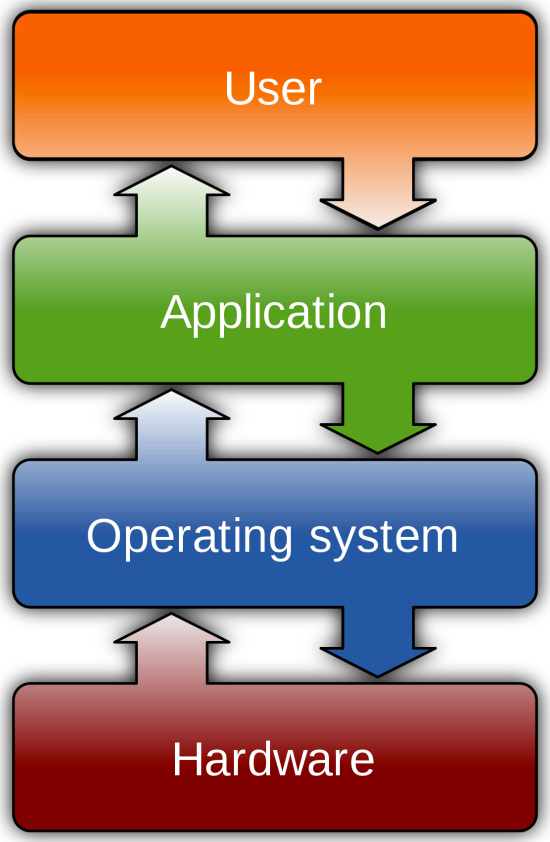
Abstraction

Layer functionality from low level (close to the hardware) to high level (close to the user).

Local, city, county, state, federal.

Each layer hides the one below it and exposes a simpler interface above.

Layers: user, application, OS, hardware



- **Applications** talk to the OS through its API, the **system calls**
- The OS talks to devices through hardware-specific code (drivers)
- "Application" is now a stack of its own: platform software such as Spark or PyTorch sits between your code and the OS

The system call is the boundary of trust.

OS v1: a library of services

Imagine the simplest possible OS: a library of routines for reading the disk and drawing the screen.

- The machine runs **one program at a time**
- No protection: we assume no bad users and no bad programs

Problem: poor utilization.

- The CPU idles while the program waits for the disk
- The human waits for each program to finish before starting the next

OS v2: multitasking

Extend the OS to hold several programs at once: when one **blocks** (disk, network, keyboard), run another.

New problem: what can an ill-behaved program do?

- Loop forever and never give the CPU back
- Scribble over another program's memory

So the OS needs two protection mechanisms:

- **Preemption**: take the CPU away from a looping program
- **Memory protection**: keep one program's memory from another

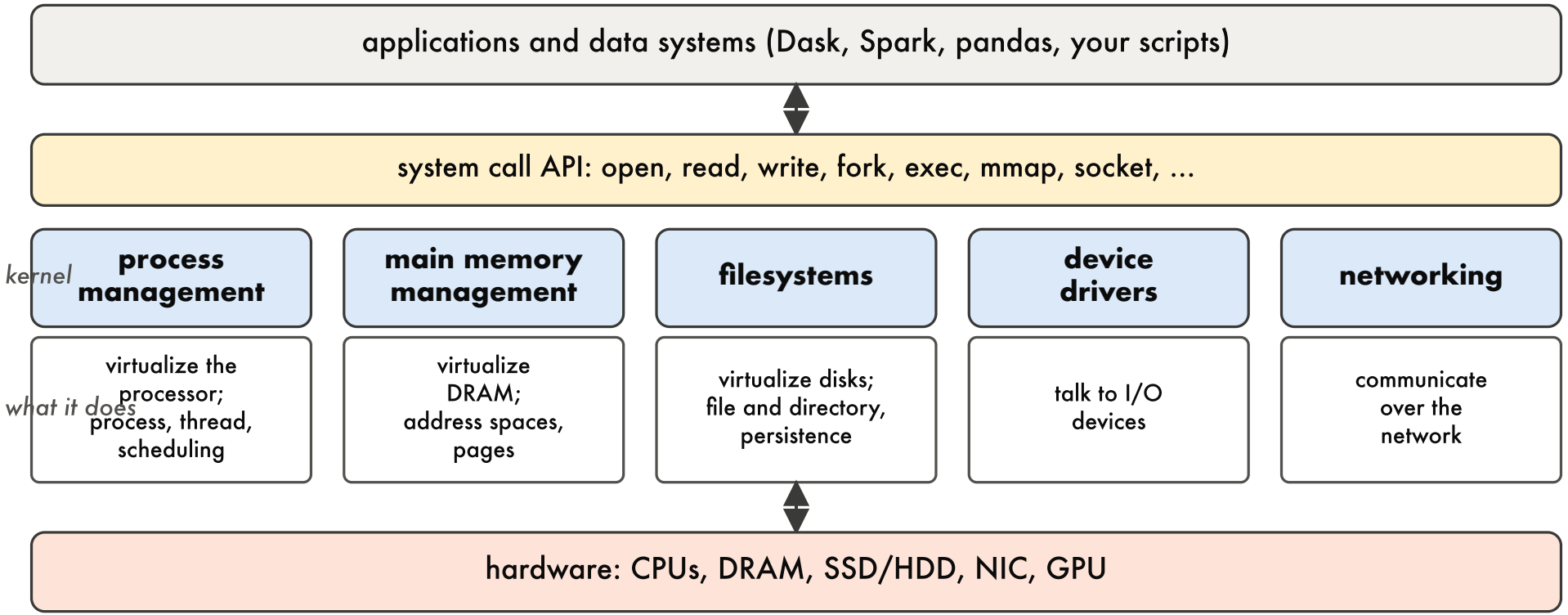
What a real OS must do

Goal: with N users or apps, the system is not N times slower. Give resources to whoever actually needs them, right now.

What can go wrong	What the OS needs
One app interferes with another	**Isolation**
Users are gluttons and hog the CPU	**Scheduling**
Total memory of all apps exceeds RAM	**Memory management**
Disks are shared by every app	**File systems**

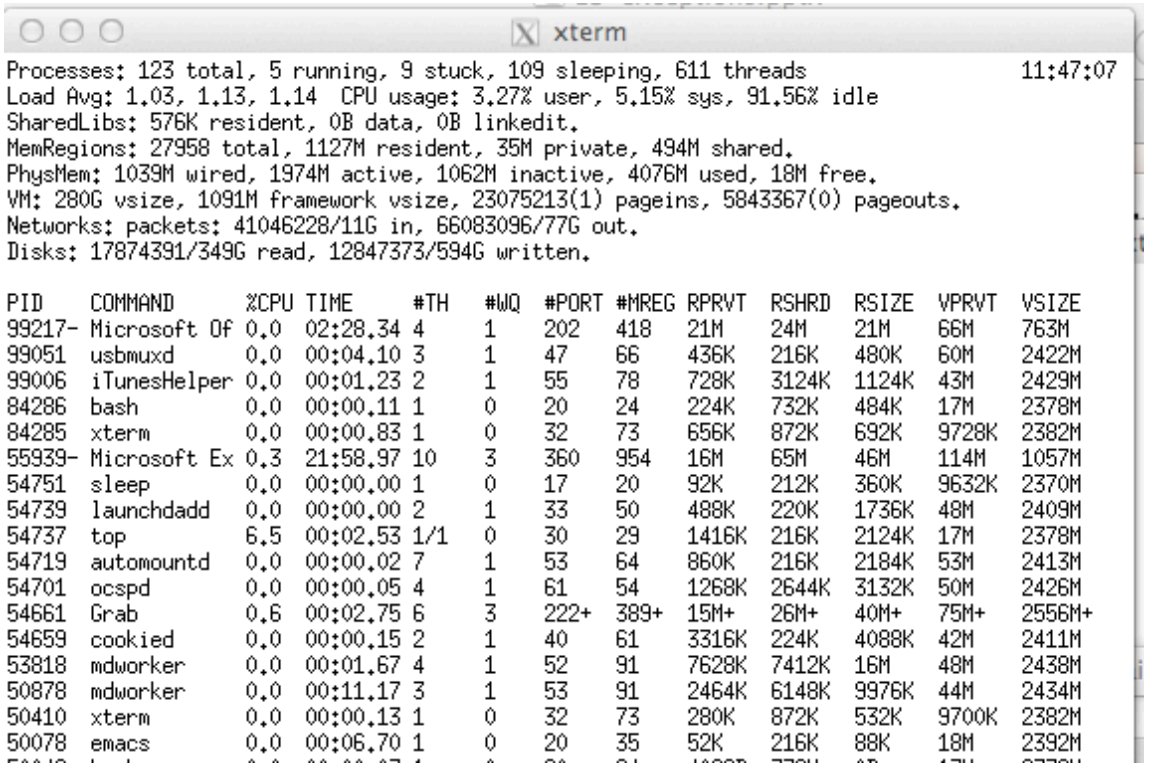
Convenience, efficiency, protection: all three, at the same time.

The kernel and its components



Kernel = the core of the OS: modules that abstract the hardware, plus the API that programs call.

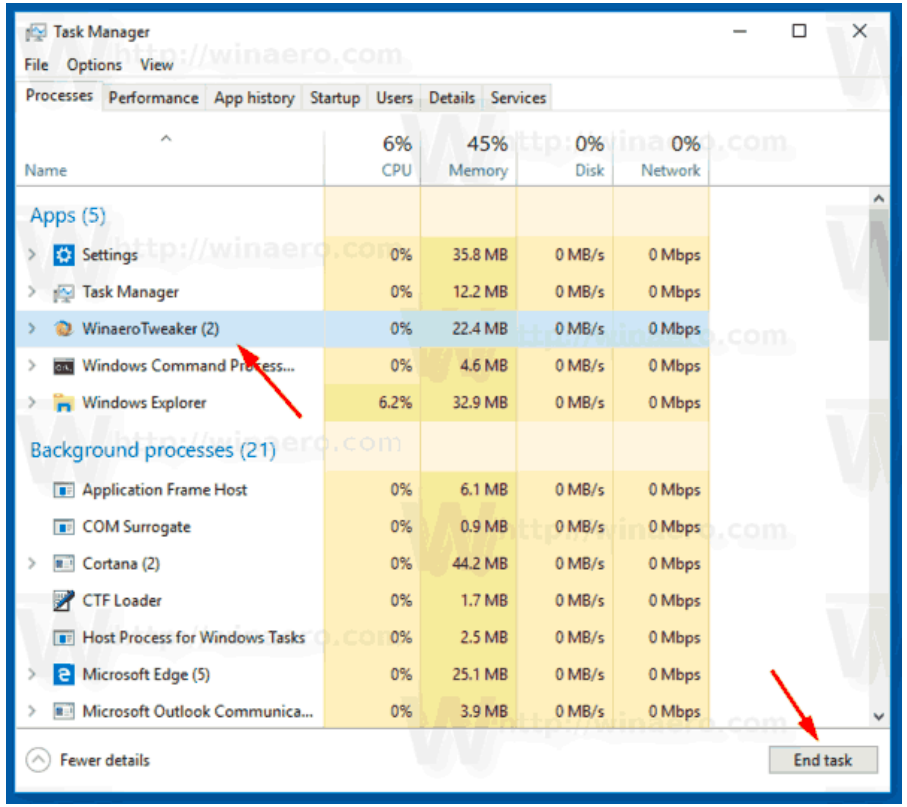
Why should a data scientist care?



- A **query is a program that becomes a process**
- Data systems (Dask, Spark, pandas) abstract process management away: you write the query, they run the processes
- In the cloud you pay per process-second, and a stuck or starved process is your problem

Knowing what the OS does under the hood tells you where the time went.

Q2 · A process is a running program



- **Process:** an instance of a program in execution, the central abstraction of the OS
- Not the **program** (a file on disk) and not the **processor** (the hardware)
- The OS keeps an inventory of live processes, the **process list**, each with a **PID**

`top`, Task Manager, and Activity Monitor all show this list.

Two illusions every process gets

"I have the CPU to myself."

The kernel mechanism: **context switching**. The CPU is shared in time, and each process is paused and resumed without noticing.

"I have the memory to myself."

The kernel mechanism: **virtual memory**. Each process gets its own address space, mapped onto shared DRAM.

Today is about the first illusion; memory comes next time.

How the OS starts a process

1. **Create** a process: assign a PID, add it to the process list
2. Assign part of DRAM to it: its **address space**
3. **Load** the code and static data into that space
4. Set up the inputs for `main()`: arguments, environment, stack
5. Mark the process **Ready**
6. When the scheduler picks it (**Running**), the OS hands the CPU to the process
7. Eventually the process finishes, or the OS **destroys** it

Steps 1 to 5 are bookkeeping; step 6 is where control leaves the kernel.

Limited direct execution

The OS hands the CPU to user code (**direct execution**, for speed) but keeps ways to take it back (**limited**).

- **Timer interrupt:** the hardware returns control to the OS every few milliseconds, no matter what the process is doing
- **System calls:** privileged operations (I/O, memory, new processes) must go through the OS
- The **scheduling policy** decides what to run after each return

Q: Is it not risky for the OS to hand the hardware to a random user-written program?

Q3 · Strawman: run each program to completion

Give each of three apps a third of the memory, and run one until it finishes, then the next.

	Convenient?	**Efficient?**	**Protected?**
Strawman	Yes	**No**	Yes

Three programs take three times as long, and the CPU idles whenever the running program waits for I/O.

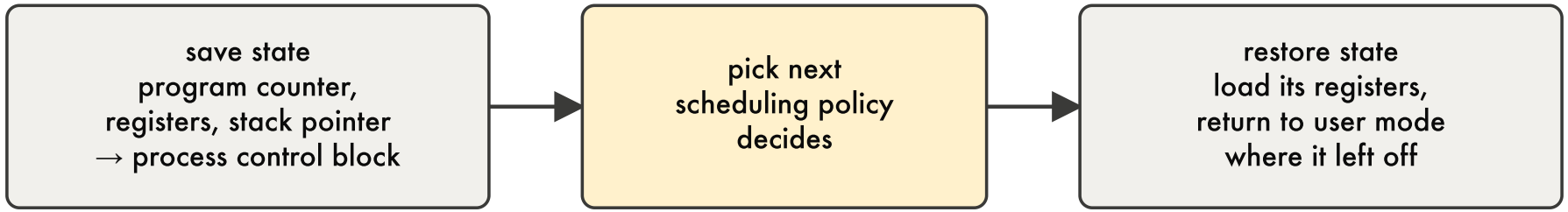
The fix is not more hardware. It is sharing the hardware in **time**.

Time sharing: the context switch

one CPU, three processes, time slices of 10 ms

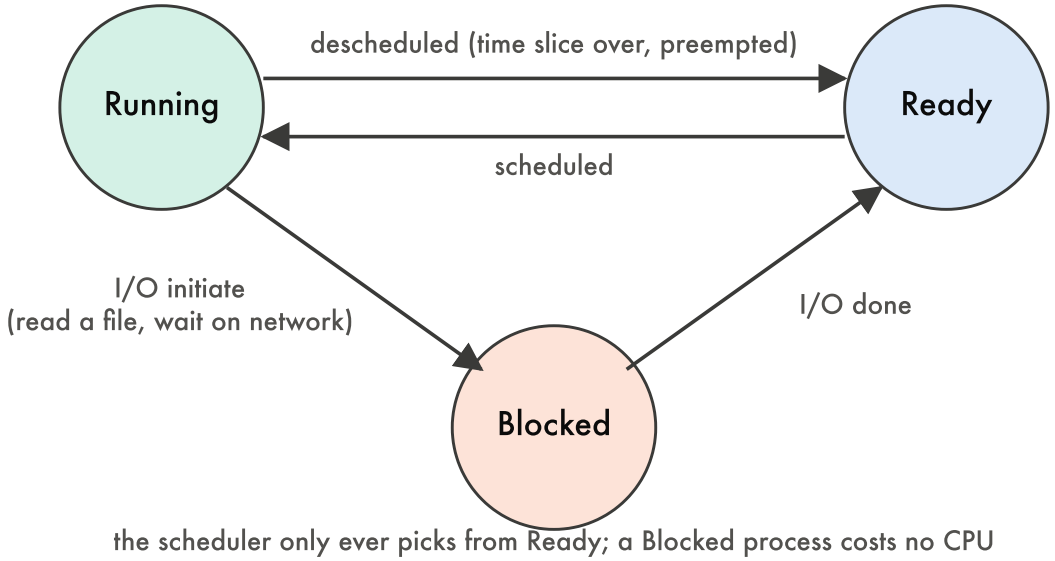


yellow slivers = context switches: save P's registers, load the next process's registers, jump



a switch takes about 1–10 µs; a 10 ms slice keeps the overhead near 0.1 %

Process states



- **Running:** on the CPU right now
- **Ready:** could run, waiting for the CPU
- **Blocked:** waiting for something else (disk, network, a lock)

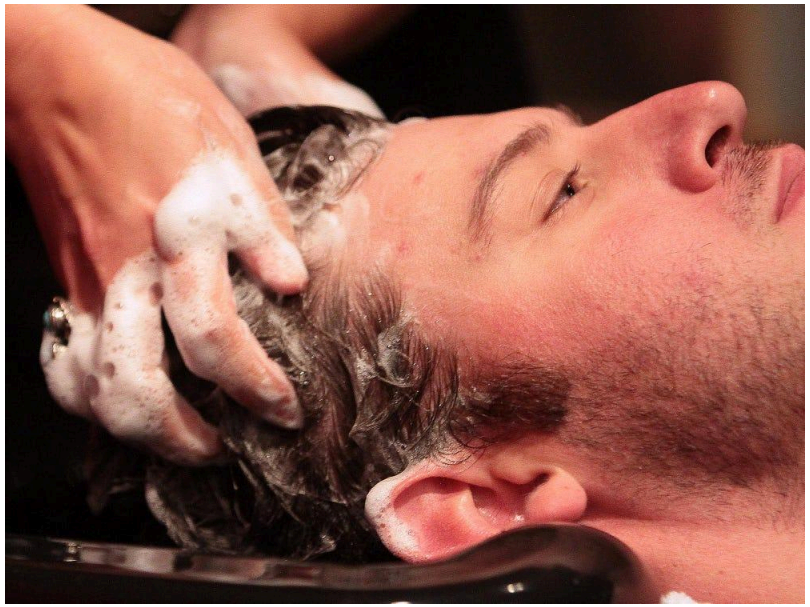
A **Gantt chart** records which process runs when:

```
P1 | idle | P2 | P1 | P2 | …
```

If a process gets stuck and nothing else is scheduled, the system hangs.

The barbershop





- The **barber** is the CPU; a **customer** is a process
- The **waiting room** is the ready queue
- A customer under a hot towel is **blocked**: the barber can serve someone else meanwhile

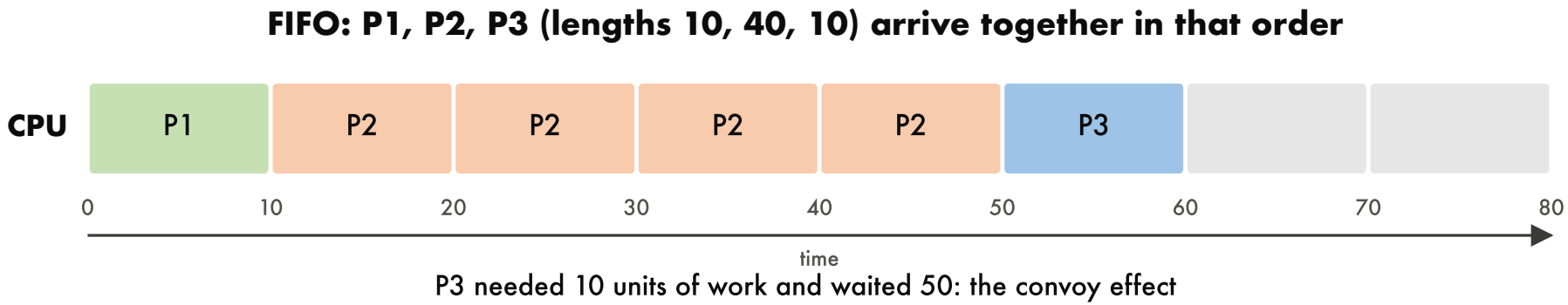
Which customer does the barber take next? That is scheduling.

Q4 · Scheduling vocabulary

Term	Meaning
Schedule	A record of which process runs on which CPU, when
Policy	The rule that produces the schedule
Arrival time	When the process is created
Job length	How much CPU time it needs
Start time	When it first gets the CPU
Completion time	When it finishes (or is killed)
Response time	Start − Arrival: how long until it begins
Turnaround time	Completion − Arrival: how long until it is done

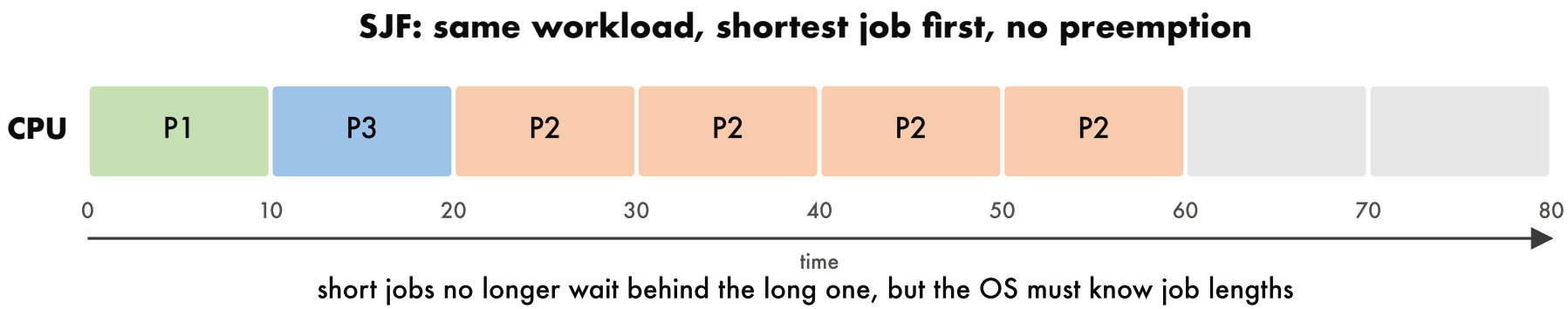
Workload: the set of processes with their arrival times and job lengths.

FIFO: first in, first out



Process	Arrival	Start	Completion	Response	Turnaround
P1	0	0	10	0	10
P2	0	10	50	10	50
P3	0	50	60	50	60
Avg				**20**	**40**

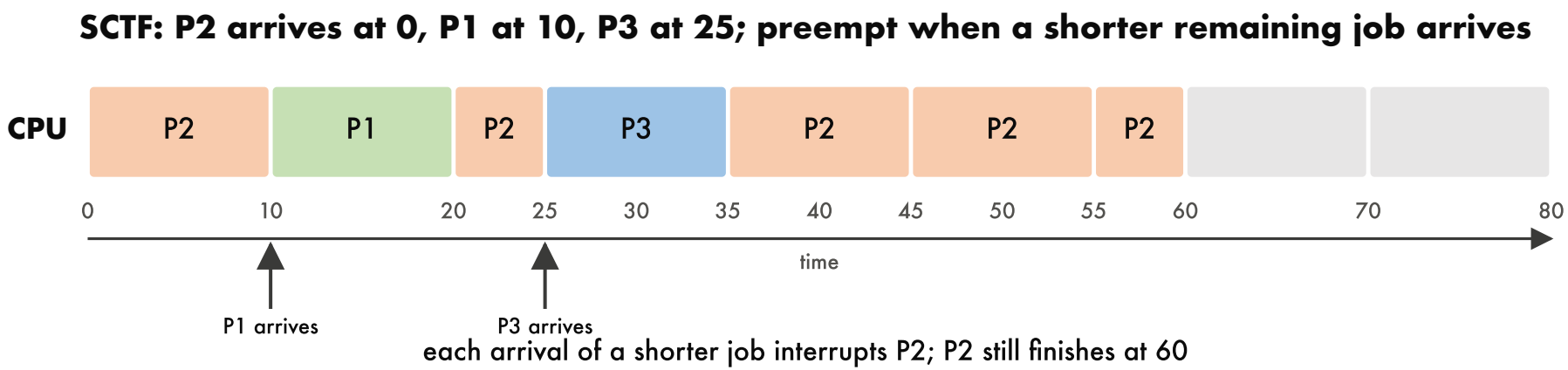
SJF: shortest job first



short jobs no longer wait behind the long one, but the OS must know job lengths

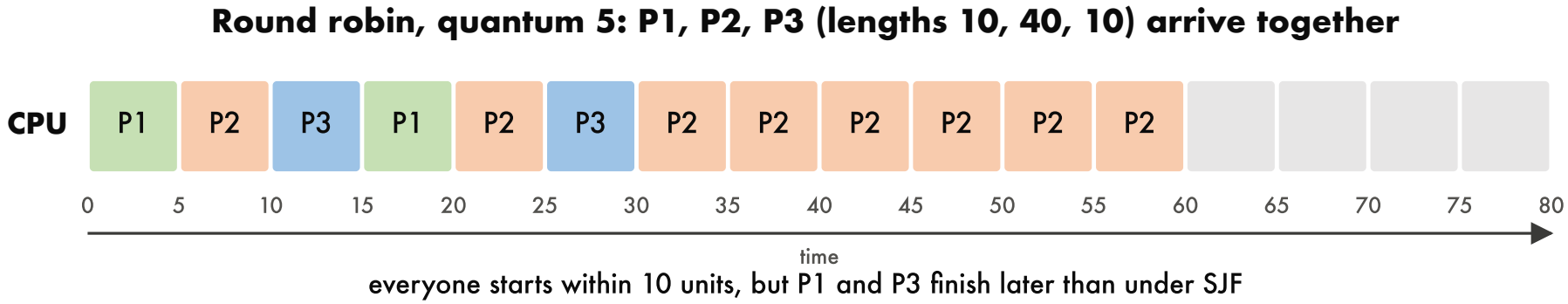
Process	Arrival	Start	Completion	Response	Turnaround
P1	0	0	10	0	10
P3	0	10	20	10	20
P2	0	20	60	20	60
Avg				**10** (FIFO: 20)	**30** (FIFO: 40)

SCTF: shortest completion time first



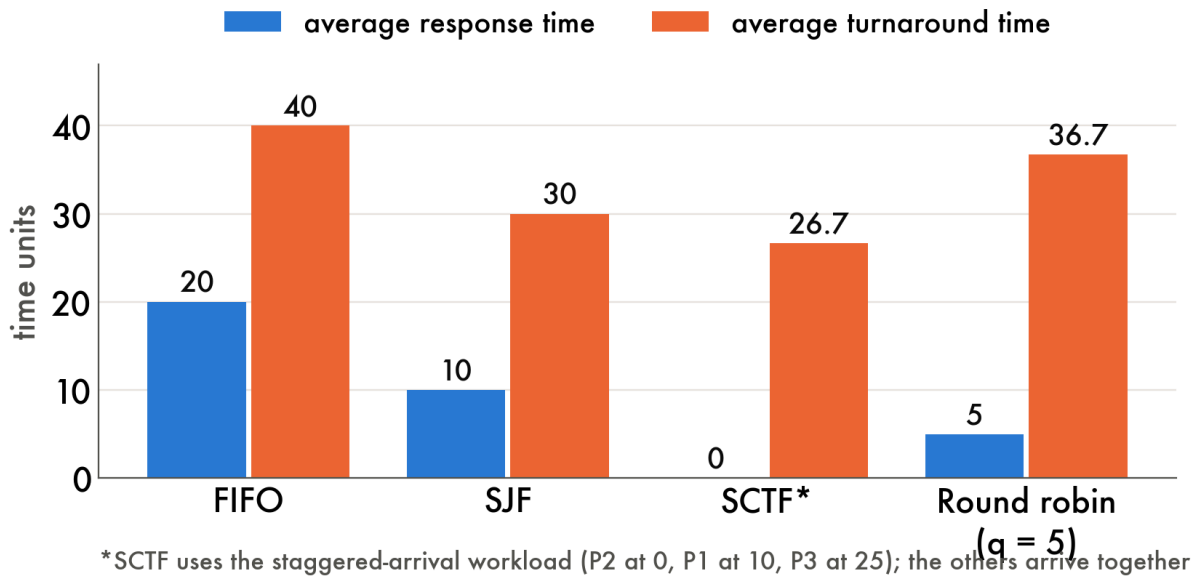
Process	Arrival	Start	Completion	Response	Turnaround
P2	0	0	60	0	60
P1	10	10	20	0	10
P3	25	25	35	0	10
Avg				**0**	**26.7**

Round robin: a fixed quantum, cycle through



Process	Arrival	Start	Completion	Response	Turnaround
P1	0	0	20	0	20
P2	0	5	60	5	60
P3	0	10	30	10	30
Avg				**5**	**36.7**

Four policies, one workload



*SCTF uses the staggered-arrival workload (P2 at 0, P1 at 10, P3 at 25); the others arrive together

- **SJF and SCTF** win on turnaround, but need job lengths
- **Round robin** needs nothing, starts everyone early, and pays for it in turnaround
- **FIFO** is simplest and worst for short jobs behind long ones

No policy wins both metrics.

Performance versus fairness

Key principle: scheduling has an inherent tension between **workload performance** and **allocation fairness**.

- Performance is usually **average turnaround time**
- Fairness has many metrics, such as **Jain's fairness index**
- In practice, arrival times and job lengths are **not known in advance**, but preemption is possible

Hundreds of policies have been studied: FIFO, SJF, STCF, round robin, random, priority, lottery.

- Real kernels use **multi-level feedback queues** that learn job behavior from history
- **ML-based schedulers** are an active research area

Scheduling in ChatGPT



Every request is a job on a shared GPU cluster.

- **Response time** = time to first token
- **Turnaround time** = the full answer
- **Job length** = the number of output tokens, unknown until the model stops
- **Fairness** = across thousands of users at once
- Can we run S1, S2, S3 **together**? Yes: batching, the GPU's version of time sharing

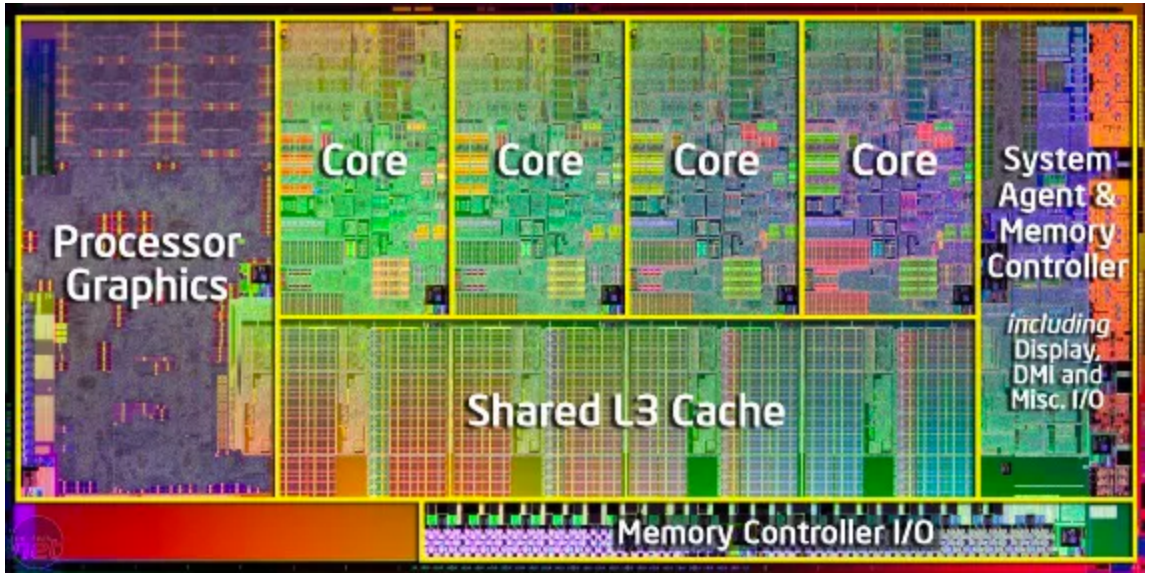
Exercise: draw the schedule

Four jobs arrive on one CPU:

Job	Arrival	Length
A	0	8
B	1	4
C	2	9
D	3	5

1. Draw the Gantt chart under **FIFO** and under **round robin with quantum 2**
2. Compute the **average response time** and **average turnaround time** for each
3. Which policy would a user of job B prefer? A user of job C?

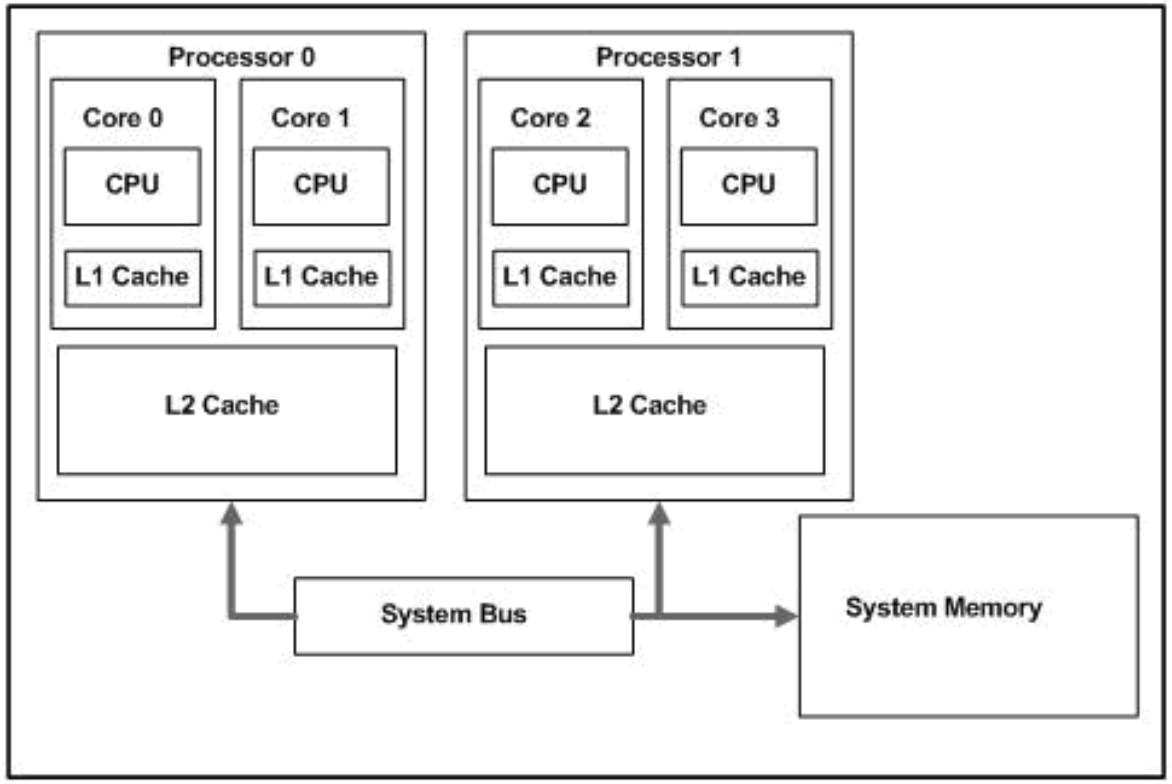
Q5 · One chip, many cores



- Modern computers have multiple processors, and multiple **cores** per processor
- **Concurrency:** cores run different (or the same) instructions **simultaneously**, on different (or shared) data
- New levels of **shared cache** appear: here, four cores share one L3

Die photo: Intel Core i7 (Sandy Bridge), Intel press materials.

The multiprocessor memory hierarchy



- Each core has **private L1 and L2** caches
- Cores on a processor share **L2 or L3**; processors share **system memory** over the bus
- Two cores can hold copies of the same line: keeping them consistent is **cache coherence**

Recall L1: the hierarchy now has a width, not just a depth.

Review questions

1. What is the purpose of an OS? Why is its design so modular?
2. What are the two illusions a process gets, and which kernel mechanism provides each?
3. Why does an OS need a scheduling policy at all?
4. Which quantity captures the latency of a process starting: response time or turnaround time?
5. What gives rise to different scheduling policies?
6. Which policy we covered is the fairest? Which has the convoy effect?
7. Explain one pro and one con of round robin over SJF.
8. Why do multi-core chips need cache coherence when single-core chips do not?

Key takeaways

1. The OS **virtualizes** the hardware: each process sees its own CPU and its own memory
2. A **process** is a running program; the OS tracks it through Ready, Running, and Blocked
3. **Time sharing** with context switches makes one CPU serve many processes
4. **Scheduling** trades workload performance against fairness; no policy wins both
5. **Many cores** add true parallelism, and with it shared caches to keep coherent